What Averages Do Not Tell

Predicting Real Life Processes with Sequential Deep Learning

István Ketykó
Eindhoven University of Technology, Mathematics and Computer Science
Eindhoven, the Netherlands
i.ketyko@tue.nl

Felix Mannhardt
Eindhoven University of Technology, Mathematics and Computer Science
Eindhoven, the Netherlands
f.mannhardt@tue.nl

Marwan Hassani
Eindhoven University of Technology, Mathematics and Computer Science
Eindhoven, the Netherlands
m.hassani@tue.nl

Boudewijn F. van Dongen
Eindhoven University of Technology, Mathematics and Computer Science
Eindhoven, the Netherlands
b.f.v.dongen@tue.nl

ABSTRACT

Process Mining concerns discovering insights on business processes from their execution data that are logged by supporting information systems. The logged data (event log) is formed of event sequences (traces) that correspond to executions of a process. Many Deep Learning techniques have been successfully adapted for predictive Process Mining that aims to predict outcomes of processes, remaining time of the process execution, the next event, or even the entire suffix of running traces. Traces are multimodal sequences and very differently structured than the natural language sentences or images often used in Deep Learning. This may require a different approach to processing. Relevant challenges may be the skewness of trace-length distribution and of the activity distribution in real-life logs. So far, there has been little focus on these differences. For suffix prediction, the performance of Deep Learning models was evaluated only on average measures and for a small number of real-life logs with different pre-processing and evaluation strategies, which makes a comparison difficult. We provide an unified end-to-end framework and compare the performance of seven state-of-the-art sequential architectures in common settings. Results show that sequence modeling still has a lot of room for improvement for the majority of the more complex datasets. Further research is required to get consistent performance not just in average measures but over all the prefixes.

CCS CONCEPTS

• **Computing methodologies** → **Neural networks**;

KEYWORDS

process mining, suffix generation, sequential deep learning

ACM Reference Format:
István Ketykó, Felix Mannhardt, Marwan Hassani, and Boudewijn F. van Dongen. 2022. What Averages Do Not Tell: Predicting Real Life Processes with Sequential Deep Learning. In *The 37th ACM/SIGAPP Symposium on Applied Computing (SAC '22), April 25–29, 2022, Virtual Event, .* ACM, New York, NY, USA, Article 4, 4 pages. https://doi.org/10.1145/3477314.3507179

SAC '22, April 25–29, 2022, Virtual Event,

ACM ISBN 978-1-4503-8713-2/22/04.
https://doi.org/10.1145/3477314.3507179

1 INTRODUCTION

Process Mining (PM) is a recent research field that advances Business Process Management (BPM) using Machine Learning (ML) and data mining solutions. Two first class citizens in this field are event logs extracted from (business) information systems, and explainable process models. Event logs contain sequential end-to-end record of what happened in the *cases* which are instances of the process. A process discovery algorithm extracts a process model out of such an event log. A process model is an explainable, graphical representation (e.g., Petri Net) of the process behavior and define which sequences of process steps (activities) are possible. A process discovery algorithm extracts such a process model from the sequential process behavior observed as event sequences (traces) in reality and as recorded in an event log.

Learning to predict how active instances or cases of a business process are likely to unfold in the future, is the main goal of predictive process mining or predictive process monitoring. Fueled by the advances made in Deep Learning (DL), the prospect of accurate predictors of the (process) future has received a lot of attention and promises organizations to act on process problems before they manifest [6]. This complements the more retrospective nature of process discovery, which provides aggregated and interpretable models of historical cases.

Several prediction tasks have been investigated in the literature [6]. The *next-event prediction* task looks at the immediate future of the process execution by predicting properties of the next event [3]. The *outcome prediction* task is concerned with what happens at the end of the process execution by predicting properties of the case, e.g., the acceptance of a loan application, or prediction of customer churn. Case or outcome properties are often concerned with some performance indicator of the process. The *suffix prediction* task attempts to construct the whole process execution by predicting properties of all future events (i.e., sequence generation). Event properties of interest include the event label or activity label and the timestamp. We focus on suffix prediction as the most challenging task for predicting event properties.

DL has been proposed for all three categories of prediction tasks including suffix prediction and consistently outperformed all alternatives based on the remaining methods. Many of the DL architectures have been adapted for event logs from Natural Language Processing (NLP) and Computer Vision, and led to substantial improvements. However, traces are multi modal, e.g., they contain both label and time, and do not form a continuous corpus as in

NLP. Additionally, the trace-length distribution of the event log can be highly skewed. DL provides efficient training due to the graphic cards and parallel processing. To ideally enable parallel processing, batching of equal-length samples is applied, which usually needs padding. Trace-length skewness leads to the necessity of introducing a large number of padding tokens.

Two studies have explored how some of the peculiarities of event logs are related to the prediction performance for next-event prediction [3] and for outcome prediction [5]. The performance of suffix prediction is evaluated only on average performance measures for a few real-life event logs, and different pre-processing and evaluation strategies are used which makes it difficult to compare the results of the different approaches [6]. Our work contributes a unified framework for case suffix prediction with seven sequential DL models, multimodal event attribute fusion, and multi-target prediction (Section 3). The end-to-end framework automatically processes event logs and avoids feature engineering. We show in detail how the performance behavior of different models vary over all prefixes of different length and highlight corresponding challenges (Section 4). We show that skewness affects all DL models and that it is insufficient to solely rely on average performance measures. An extended version of this paper with an additional set of experiments is available in [4].

2 PROBLEM FORMULATION AND NOTATION

In the suffix and remaining time prediction, the dataset, i.e., event log, is a set of sequences (process executions or *traces*) $D = \{\sigma^{(1)}, \ldots, \sigma^{(d)}\}$, where d is the size of dataset. The i-th process execution $\sigma^{(i)} = \langle e_1, \ldots, e_n \rangle$ contains a sequence of n events, i.e., $|\sigma^{(i)}| = n$. An event $e_j = (a_j, t_j)$ has two attributes where the former is the event's label, i.e., an activity, and the latter is the event's duration time, or the required execution time. For a given process execution $\sigma^{(i)} = \langle (a_1, t_1), \ldots, (a_n, t_n) \rangle$, the prefix of events of length k is defined by $\sigma^{(i)}_{\leq k} = \langle (a_1, t_1), \ldots, (a_k, t_k) \rangle$ and its corresponding suffix of events is $\sigma^{(i)}_{>k} = \langle (a_{k+1}, t_{k+1}), \ldots, (a_n, t_n) \rangle$.

Definition 2.1 (Suffix prediction and remaining time prediction). Suppose that there are pairs sample of sequences $\mathcal{S} = \{(\sigma^{(i)}_{\leq k}, \sigma^{(i)}_{>k})\}_{i=1}^{i=n}$, where $2 \leq k < |\sigma^{(i)}|$ is the prefix length and n is the sample size (also the set of prefixes $\mathcal{P} = \{\sigma^{(i)}_{\leq k}\}_{i=1}^{i=n}$). Given a prefix of events sequence, the output prediction is the sequence of events $\hat{\sigma}_{>k} = \langle (a_{k+1}, t_{k+1}), \ldots, \texttt{[EOS]} \rangle$, where `[EOS]` is a special symbol added to the end of each process execution to mark the end of the sequence in prepossessing time. Suffix prediction is the sequence of activities in $\hat{\sigma}_{>k}$, i.e., $\langle a_{k+1}, \ldots, \texttt{[EOS]} \rangle$. The remaining time prediction is the sum of the predicted duration time t in $\hat{\sigma}_{>k}$, i.e., $\sum t_i$, where $t_i \in \hat{\sigma}_{>k}$.

3 PROPOSED FRAMEWORK

The proposed framework in Fig. 1 provides deep, autoregressive modeling for multi-objective prediction tasks of multimodal sequences. Our vision is to perform no feature engineering and to use the original event logs without excluding traces or trace parts.

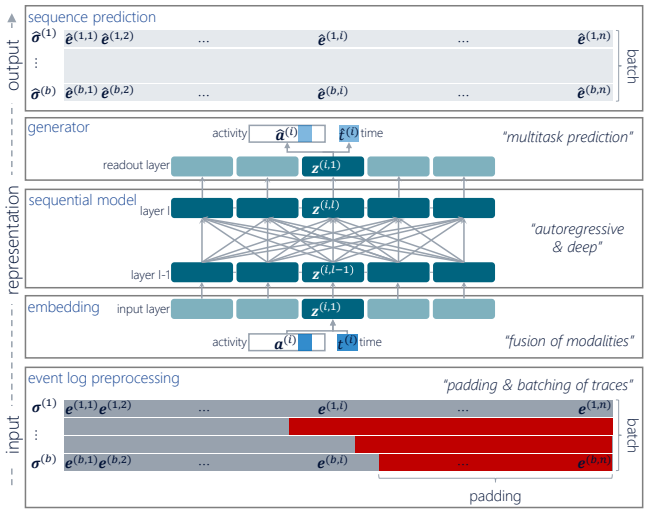

Figure 1: Our unified framework for sequential DL model comparisons for suffix generation.

We describe the three main components of our framework: embedding of an input sequence for the fusion of modalities (Section 3.1), the pluggable sequential model component (Section 3.2), and the generator providing multitask predictions (Section 3.3).

3.1 Embedding

In the proposed framework each event $e_i = (a_i, t_i)$ is shown by tuple $\mathbf{e}^{(i)} = (\mathbf{a}^{(i)}, t_i)$, where $\mathbf{a}^{(i)}$ is the one-hot encoding of activity a_i. We denote the j-th entry of a vector by the corresponding subscript, e.g., $\mathbf{a}^{(i)}_j$. The embedding component maps the input features $\mathbf{e}^{(i)}$ into the latent space of the model.

The heterogeneous event attributes have to be represented in form which is adequate as input features for machine learning computations. The categorical attributes (i.e. activity label) are one-hot encoded into a vector $\mathbf{a}^{(i)}$, and the continuous attributes (i.e. timestamp) are min-max scaled to the range of $[0, 1]$ and represented with a scalar $t^{(i)}$. Our framework uses two attributes: the activity label, and the timestamp. We take the time attribute on the scale of seconds granularity and transform it to relative time expressing the duration of events (by the difference of two consecutive event timestamps). This also prevents data leakage. The time attribute value corresponding to any special symbols for the activity label (e.g. `[EOS]`, `[PAD]`, `[SOS]`, and `[MASK]`) is defined as zero. For utilising parallel processing resources, the input sequences are batched. This introduces a constraint: sequences in a batch have to be of equal length. Several approaches are possible to ensure that, such as splitting the original sequences into smaller moving-windows of sub-sequences or left/middle/right padding of the original sequences. All approaches may impact the prediction performance of the model: windowing limits the receptive field of the model and padding changes the activity label distribution of the dataset due to the excessive amount of padding needed.

The event attributes have to be fused. We apply learnable linear transformations for each feature then vector summation.

3.2 Sequential Deep Learning Models

We introduce ML with special emphasis on generative and AutoRegressive (AR) modeling, then we detail several DL architectures that we tested in the framework.

We describe 7 DL architectures (out of which 4 are first time applied on suffix generation in PM) that all model autoregression but are built up with different building blocks altering training and inference (computation and memory) complexity, model (parameter) size, perspective view (along the sequence), path length (between two positions in the sequence), and parallelisability of operations during training. We use the **1)** Long Short Term Memory (**LSTM**) network [1], **2)** Sequential AutoEncoder (**AE**) [8], **3)** Sequential AE with an auxiliary adversarial objective (**AE-GAN**) [8], **4)** Self-attentional **Transformer** [10], **5)** Generative Pre-Training (**GPT**) Transformer Decoder [7], **6)** Bidirectional Encoder Representations from Transformers (**BERT**) with Probabilistic Masked Language Modeling (PMLM) [2], **7)** Dilated causal (1-D) Convolutional Neural Network (**WaveNet**) [9]. The architectures are detailed in [4].

3.3 Generator

The generator component offers multitask predictions. For each event in the sequence, categorical and numerical variables are predicted, respectively. For the categorical *activity label* variable, the readout provides a vector of logits $\hat{\mathbf{a}}^{(\mathbf{i})}$. The logits are transformed into likelihoods by the softmax function. There are several methods to decode the discrete token out of this categorical distribution of $\hat{\mathbf{a}}^{(\mathbf{i})}$. The most commonly used method is to select the most likely category, that is also called as argmax or greedy search. Beam search, an algorithmic breadth-first search technique, can yield better quality of sequences with the cost of increased memory and computation. Different forms of stochastic sampling (from the categorical distribution) can be used also. We use the most common greedy search solution for the experiments.

The proposed framework provides multi-objective optimisation. Learning for the categorical features is via the categorical cross-entropy loss (i.e. of the ground truth $\mathbf{a}^{(\mathbf{i})}$ and the predicted $\hat{\mathbf{a}}^{(\mathbf{i})}$): $\mathcal{L}_{\text{cross-entropy}} = -\mathbf{a}^{(\mathbf{i})} \cdot \log \hat{\mathbf{a}}^{(\mathbf{i})}$. The loss function is the average of such errors over all items in the sequence. Learning for the continuous features is via the squared error (i.e. of the ground truth $t^{(i)}$ and the predicted $\hat{t}^{(i)}$): $\mathcal{L}_{\text{squared error}} = (\hat{t}^{(i)} - t^{(i)})^2$. The loss function is the average of such errors over all items in the sequence.

The final loss is a weighted sum of the individual losses. During inference the sequence generation is in a loop; a step is always conditioned on the previously generated context up to the `[EOS]` or predefined maximum length. We set that maximum to the length of the longest trace in the dataset which is a pragmatic choice.

4 EXPERIMENTAL RESULTS

4.1 Evaluation Measures

To evaluate the performance of suffix prediction (in the viewpoint of the sequence of categorical activity labels), we use Damerau-Levenstein distance (DL). We normalise it and take the inverse of it to end up with Damerau-Levenstein Similarity (DLS) score. Also, we compute the absolute error between the ground truth remaining time and the predicted remaining time for each predicted and ground truth suffixes. Next, we average these numbers for evaluation instances, and report Mean Absolute Error (MAE).

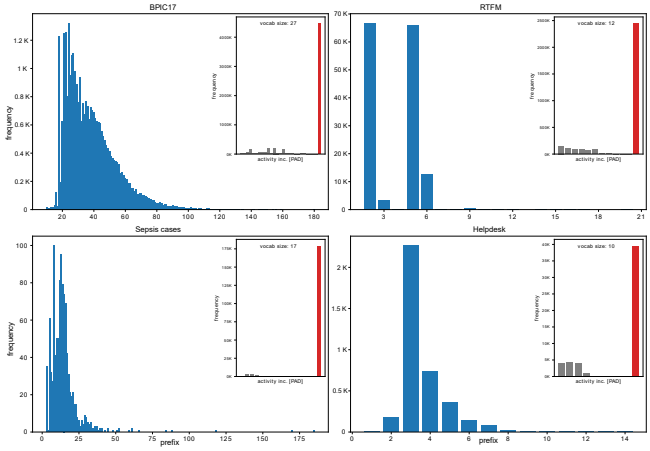

Figure 2: Case length and activity label (including the [PAD] symbol) distributions (full version in [4]).

4.2 Datasets

We evaluate the performance of the models on 9 real-life datasets (which are very commonly used in the PM community)[1] without prepossessing. Fig. 2 shows the case length and activity distributions for all the datasets. The case length distribution of all datasets is heavily skewed. This brings a challenging situation for sequential prediction tasks. The models have to represent the underrepresented long traces, in the long tail of the distributions. The embedded activity distribution plots include the added `[PAD]` special symbol (red bar) and show the extreme relative frequency of `[PAD]`. We apply splitting into training and evaluation sets by 8:2 ratio after shuffling of traces, and use the 80% of the traces for training, and the remaining 20% traces for evaluation. We use the exact same subsets for all experiments.

4.3 Experimental Setup

The proposed framework is implemented[2] in Python 3.7, PyTorch 1.9, and CUDA 10.2. Throughout all experiments we keep an equal number of 4 layers with 128 latent vector size of the sequential component, plus an embedding and generator with the same latent size. We train models for 400 epochs and apply early stopping if we observe no more improvement on the evaluation set for 50 iterations. We use Adam optimisation with learning rate 1e−4. To improve on generalisation we apply a dropout rate of 0.3.

4.4 Results

As Table 1 shows all the models except BERT perform on a comparable level. The reason might be that BERT is more sensitive to the skewness because of its non-AR training objective. Furthermore, masked language modeling (PMLM) might require more training

[1] Publicly available at https://data.4tu.nl/search?q=:keyword:%20%22Task%20Force%20on%20Process%20Mining%22.
[2] http://github.com/smartjourneymining/sequential-deep-learning-models

Table 1: Average DLS results (best and *worst* performers per dataset).

model/dataset	AE↑	WaveNet↑	BERT↑	GPT↑	LSTM↑	Transformer↑	AE-GAN↑
BPI17	**0.1417**	0.1326	*0.0306*	0.1412	0.0593	0.0931	0.0697
BPI15 municipality 1	**0.4035**	0.2393	*0.0000*	0.1919	0.0276	0.3356	0.2375
BPI12	**0.1737**	0.1518	*0.0333*	0.1647	0.0677	0.1004	0.1666
BPI13 closed problems	**0.6820**	0.6729	*0.0000*	0.4236	0.4721	0.5762	0.6271
BPI13 incidents	**0.4363**	0.2894	*0.0057*	0.2859	0.2610	0.1631	0.2863
BPI13 open problems	0.1562	**0.4372**	*0.0000*	0.3879	0.3703	0.1887	0.2378
RTFM	0.8236	0.8218	0.8107	**0.8263**	*0.3708*	0.8230	0.7975
Sepsis cases	0.2169	0.0892	*0.0000*	0.0924	**0.2218**	0.0391	0.1396
Helpdesk	0.8593	0.8599	*0.0000*	**0.8609**	0.8599	0.8471	0.8599

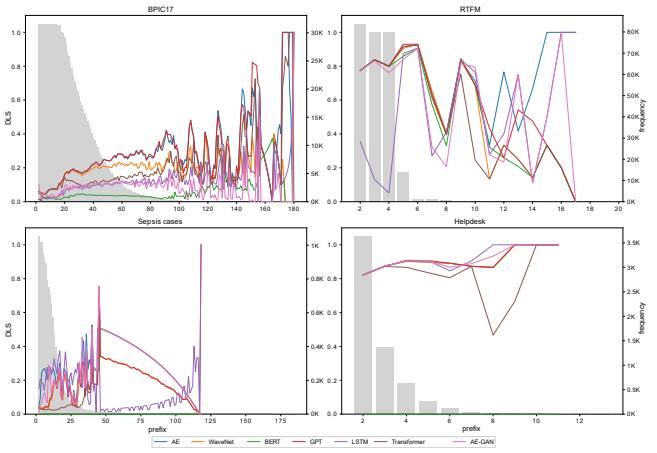

Figure 3: DLS and frequency values over all prefix combinations per dataset (full version in [4]).

epochs (since it approximates all perturbations) for suffix prediction. AR models perform well on datasets with shorter traces and smaller vocabulary such as the Helpdesk and RTFM logs. Those logs are also relatively more much structured. BPI17 and Sepsis cases appear to be the most challenging datasets. Together with BPI12, those also have the longest traces.

Fig. 3 visualises the DLS of the suffix generations results for different prefix lengths (x-axis) starting from length two. We visualise the performance change of the model with a line chart (left y-axis) and the frequency of prefixes of a certain length in the data with a bar chart (right y-axis). The intuition would be that by increasing prefix size the DLS generally increases but it is empirically different. The bar chart aims to shed the light of the underlying phenomena. It could be connected to the number of traces in the dataset which are equal or longer than the given prefix length. Due to the non-uniform trace length distribution, the count of traces is monotonically decreasing given the increasing prefix length. The slope of that decrease is small for shorter prefixes: in that interval the DLS generally increases. That is generally followed by a large drop in the count possibly due to the skewness: that results in performance degradation of the models in general. The models tend to be biased towards predicting shorter suffixes because there are much less longer traces in the dataset. The DLS calculation heavily punishes that bias. However, there are perfect DLS scores for the prefix sizes which are very close to the longest trace(s). That may be the result of overfitting given the infinitesimal variance among those extremely few traces. It is interesting that there are results missing for some prefix lengths even though these prefixes do exist in the data (bar chart). This is due to the training-evaluation random split: in the evaluation subset there are no corresponding traces with the same length. Thus, a sole, average DLS metric seems to be not always informative enough. For example, for the RTFM dataset GPT has the highest average DLS but for the prefix length of 12, it is the worst performing.

5 CONCLUSIONS

We investigated how 7 sequential DL architectures (out of which 4 have never been applied on suffix generation in PM) perform for predicting the suffix of multi-modal sequences. Evaluating the architectures across all prefix lengths showed that none of them is one-size-fits-all and that with increasing lengths, the prediction performance for some prefixes fluctuates heavily with substantial drops in performance. These results have implications on how performance of DL models should be reported. Solely reporting the average DLS for suffix prediction seems to be inadequate for comparing DL architectures and for assessing their suitability in practice. Our hypothesis is that this effect is due to the skewness of the length distribution for traces in event log, which prevents the direct application of successful DL architectures for NLP tasks (in particular, BERT) in our experimental setting. In future work, also the impact of other event log properties should be investigated for suffix prediction. Furthermore, controlled experiments (e.g., on simulated data from process models could be carried out to identify the impact of specific data properties. Finally, our experiments should be extended with hyperparameter tuning for each architecture to avoid the influence of manual parameter choices.

Acknowledgement. This work is part of the Smart Journey Mining project, which is funded by the Research Council of Norway (project no. 312198).

REFERENCES

[1] Manuel Camargo, Marlon Dumas, and Oscar González-Rojas. 2019. Learning Accurate LSTM Models of Business Processes. In *Business Process Management*. Springer International Publishing, Cham, 286–302.
[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL*. 4171–4186.
[3] Kai Heinrich, Patrick Zschech, Christian Janiesch, and Markus Bonin. 2021. Process data properties matter: Introducing gated convolutional neural networks (GCNN) and key-value-predict attention networks (KVP) for next event prediction with deep learning. *Decis Support Syst* 143 (2021), 113494.
[4] István Ketykó, Felix Mannhardt, Marwan Hassani, and Boudewijn van Dongen. 2021. What Averages Do Not Tell – Predicting Real Life Processes with Sequential Deep Learning. (2021). arXiv:cs.LG/2110.10225
[5] Wolfgang Kratsch et al. 2021. Machine Learning in Business Process Monitoring: A Comparison of Deep Learning and Classical Approaches Used for Outcome Prediction. *BISE* 63 (2021), 261–276.
[6] Dominic A. Neu, Johannes Lahann, and Peter Fettke. 2021. A systematic literature review on state-of-the-art deep learning methods for process prediction. *Artificial Intelligence Review* (2021).
[7] Alec Radford and Karthik Narasimhan. 2018. Improving Language Understanding by Generative Pre-Training.
[8] Farbod Taymouri, Marcello La Rosa, and Sarah M. Erfani. 2021. A Deep Adversarial Model for Suffix and Remaining Time Prediction of Event Sequences. In *SDM21*. 522–530.
[9] Aaron van den Oord et al. 2016. WaveNet: A Generative Model for Raw Audio. In *Arxiv*. https://arxiv.org/abs/1609.03499
[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, Vol. 30.